

Introduction to Spring AOP

Aspect-Oriented Programming (AOP):

- **Definition:** A programming paradigm that allows the separation of cross-cutting concerns (e.g., logging, security).
- **Purpose:** Solves problems related to cross-cutting concerns without scattering them across the codebase.
- **Benefits:** Simplifies maintenance by allowing you to create centralized concerns that can be invoked automatically.

Problem:

- Without AOP, cross-cutting concerns like logging and security checks are spread across multiple parts of the application, leading to code duplication.

Solution:

- Use AOP to create aspects that are called automatically to handle cross-cutting concerns.

Logging the Calls

- **Task:** Create a `LoggingAspect` class to log method calls within controllers.
- **Tools Used:**
 - `LoggerFactory` for logging.

Example:

```
@Aspect
@Component
public class LoggingAspect {
    private static final Logger LOGGER = LoggerFactory.getLogger(LoggingAspect.class);

    @Before("execution (* com.telusko.springbootrest.service.JobService.*(..))")
    public void logMethodCall() {
        LOGGER.info("Method Called ");
    }
}
```

- **Explanation:** The `@Before` advice logs any method execution within the specified package. This is a starting point for integrating logging.

AOP Concepts

Terminologies:

- **Aspect:** A module that encapsulates behaviors affecting multiple classes.
- **Join Point:** A point in the execution of a program, such as method execution or exception handling.
- **Advice:** The action taken by an aspect at a particular join point (e.g., `@Before`, `@After`).
- **Pointcut:** The expression that defines at which join points advice should be applied.
- **Target:** The object being advised by one or more aspects.
- **AOP Proxy:** A proxy object created by the AOP framework.
- **Weaving:** The process of linking aspects with other application types.

AOP Concepts in a Nutshell

1. Join Point (When):

- **Movie Analogy:** A scene in the movie where the action happens.
- **AOP Definition:** The specific point in the program's execution where the action (advice) will be applied. This could be method execution, exception handling, etc.

2. Advice (What):

- **Movie Analogy:** The action that happens at a particular scene; the plot twist.
- **AOP Definition:** The code that is executed at a join point. It defines the "what" of the AOP implementation. Examples include before advice, after advice, and around advice.

3. Aspect (Where - Conceptual):

- **Movie Analogy:** The script of your movie; it defines what plots happen and where.

- **AOP Definition:** A module that captures cross-cutting concerns (like logging, security) and applies them to other modules using pointcuts. It defines the "where" of the AOP implementation.

4. Pointcut (Where - Operational):

- **Movie Analogy:** The specific scenes (join points) where the plot twists (advice) occur, like a bookmark in your script.
- **AOP Definition:** An expression that defines the join points where the advice should be applied. It's the operational definition of where the advice will be executed.

5. Target Object (Whom):

- **Movie Analogy:** The main character who experiences the plot twists (advice).
- **AOP Definition:** The object that the advice is applied to. It's the object that the advice will affect.

6. Weaving (How):

- **Movie Analogy:** The director's job; how the script (aspect) is turned into a movie.
- **AOP Definition:** The process of linking the aspect with the target object to create the advised object. This can happen at compile time, load time, or runtime.

7. Proxy (The Double):

- **Movie Analogy:** The stunt double; takes the hits and makes the main character look good.
- **AOP Definition:** An object that is created to represent the target object after the advice has been applied. The proxy intercepts method calls to the target object and applies the advice before or after the actual method execution.

8. Types of Advice (The Genre):

- **Movie Analogy:** The genre of your movie; the style of how the plot twists (advice) will unfold.
- **AOP Definition:** Different types of advice in Spring AOP:
 - **Before:** Executed before the method execution.
 - **After:** Executed after the method execution, regardless of the outcome.
 - **After-returning:** Executed after the method execution if it completes successfully.
 - **After-throwing:** Executed after the method execution if it throws an exception.
 - **Around:** Executes both before and after the method execution, giving you control over the entire method invocation.

Before Advice

Annotations:

- **@Before**: Executes advice before the matched method runs.
- Annotation Used: **@Before("pointcut expression")**

Expression Example:

```
@Component
@Aspect
public class LoggingAspect{

    private static final Logger LOGGER =
        LoggerFactory.getLogger(LoggingAspect.class);

    @Before("execution (* com.telusko.springbootrest.service.JobService.*(..))")
    public void logMethodCall() {
        LOGGER.info("Method Called ");
    }
}
```

- **Explanation:**

- This allows logging or performing actions before method executions without modifying the target method (**JobService**).

JoinPoint

- **Pointcut Expression:** Defines when advice runs.
- **JoinPoint:** Provides access to method signatures and arguments.

Example:

```
//@Before("execution (* com.telusko.springbootrest.service.JobService.getJob(..) ||  
           execution(* com.telusko.springbootrest.service.JobService.updateJob(..))")  
  
@Before("execution (* com.telusko.springbootrest.service.JobService.getJob(..)")  
public void logMethodCall(JoinPoint jp) {  
    LOGGER.info("Method Called "+jp.getSignature().getName());  
}
```

After Advice

Annotations:

- **@After**: Runs after the method finishes (regardless of outcome).
- **@AfterThrowing**: Runs after a method throws an exception.
- **@AfterReturning**: Runs after a method returns successfully.

Examples:

```
@Aspect
@Component
public class LoggingAspect {

    public static final Logger LOGGER=LoggerFactory.getLogger(LoggingAspect.class);

    @Before("execution (* com.telusko.springbootrest.service.JobService.getJob(..) ||
        execution(* com.telusko.springbootrest.service.JobService.updateJob(..))")
    public void logMethodCall(JoinPoint jp) {
        LOGGER.info("Method Called "+jp.getSignature().getName());
    }

    @After("execution (* com.telusko.springbootrest.service.JobService.getJob(..) ||
        execution(* com.telusko.springbootrest.service.JobService.updateJob(..))")
    public void logMethodExecuted(JoinPoint jp) {
        LOGGER.info("Method Executed "+jp.getSignature().getName());
    }

    @AfterThrowing("execution (* com.telusko.springbootrest.service.JobService.getJob(..) ||
        execution(* com.telusko.springbootrest.service.JobService.updateJob(..))")
    public void logMethodCrashed(JoinPoint jp) {
        LOGGER.info("Method has some issues "+jp.getSignature().getName());
    }

    @AfterReturning("execution (* com.telusko.springbootrest.service.JobService.getJob(..) ||
        execution(* com.telusko.springbootrest.service.JobService.updateJob(..))")
    public void logMethodExecutedSuccess(JoinPoint jp) {
        LOGGER.info("Method Executed Successfully "+jp.getSignature().getName());
    }
}
```

Performance Monitoring with Around Advice

- **Task:** Monitor method execution time.
- **Annotation:** `@Around`

Example:

```
@Component
@Aspect
public class PerformanceMonitorAspect {

    public static final Logger = LOGGER
    LoggerFactory.getLogger(PerformanceMonitorAspect.class);

    @Around("execution (*com.telusko.springbootrest.service.JobService.*(..))")
    public Object monitorTime(ProceedingJoinPoint jp) throws Throwable {

        long start=System.currentTimeMillis();

        Object obj= jp.proceed();
        long end=System.currentTimeMillis();

        LOGGER.info( "Time taken by: " +jp.getSignature().getName()+"
                    " +(end-start)+ " ms");
        return obj;
    }
}
```

Explanation:

- `@Around` advice wraps method calls, enabling actions before and after execution.

Input Validation Using Around Advice

- **Task:** Validate method input.

Example:

```
@Component
@Aspect
public class ValidationAspect {
    public static final Logger LOGGER=
        LoggerFactory.getLogger(ValidationAspect.class);

    @Around("execution (*com.telusko.springbootrest.service.JobService.getJob(..)) &&
        args(postId)")
    public Object validateAndUpdate(ProceedingJoinPoint jp,int postId) throws Throwable {
        if (postId<0) {
            LOGGER.info("PostId is negative, updating it");

            postId=-postId;
            LOGGER.info("new Value "+postId);
        }

        Object obj=jp.proceed(new Object[] {postId});

        return obj;
    }
}
```

Explanation:

- The `&& args(posId)` expression ensures the advice runs only if `posId` is passed as an argument.